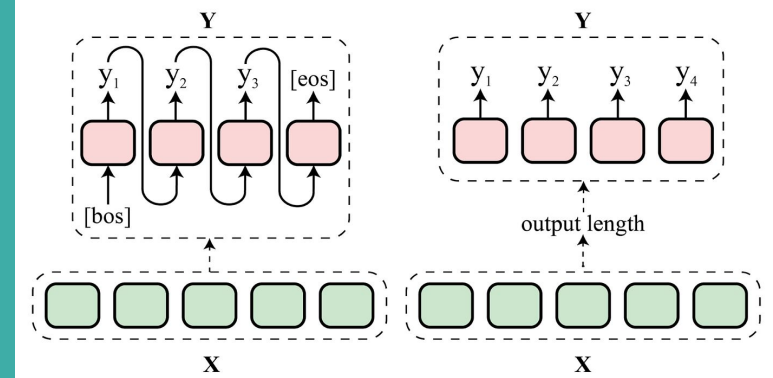


Deep Learning

CSE641/ECE555

Lecture 20 | Take your own notes during lectures

Vinayak Abrol <abrol@iiitd.ac.in>



INDRAPRASTHA INSTITUTE of
INFORMATION TECHNOLOGY DELHI

Auto-Regression

Auto-regression methods model components of a signal serially, each one conditionally to the ones already modeled.

They rely on the chain rule from probability theory: given $X_1, \dots, X_T$ random variables, we have

$$P(X_1 = x_1, \dots, X_T = x_T) = P(X_1 = x_1)P(X_2 = x_2|X_1 = x_1) \dots P(X_T = x_T|X_1 = x_1, \dots, X_{T-1} = x_{T-1})$$

Auto-Regression

Auto-regression methods model components of a signal serially, each one conditionally to the ones already modeled.

They rely on the chain rule from probability theory: given $X_1, \dots, X_T$ random variables, we have

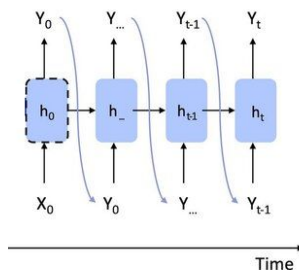
$$P(X_1 = x_1, \dots, X_T = x_T) = P(X_1 = x_1)P(X_2 = x_2|X_1 = x_1) \dots P(X_T = x_T|X_1 = x_1, \dots, X_{T-1} = x_{T-1})$$

Larochelle and Murray, 2011

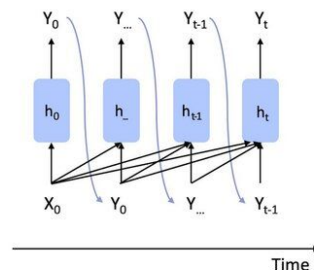
DNNs in large dimension setting decomposes the joint distribution of observations into tractable conditional distributions and models each conditional using a nonlinear function.

It can also be interpreted as an autoencoder wired such that its output can be used to assign valid probabilities to observations.

Recurrent Neural Network



Autoregressive Model



Auto-Regression: Masking

Given a sequence of random variables, we can represent a conditioning event with two tensors of dimension T : the first a Boolean mask stating which variables are conditioned, and the second the actual conditioning values.

E.g., with $T = 5$

Event	Mask tensor	Value tensor
$\{X_2 = 3\}$	$[0, 1, 0, 0, 0]$	$[0, 3, 0, 0, 0]$
$\{X_1 = 1, X_2 = 2, X_3 = 3, X_4 = 4, X_5 = 5\}$	$[1, 1, 1, 1, 1]$	$[1, 2, 3, 4, 5]$
$\{X_5 = 50, X_2 = 20\}$	$[0, 1, 0, 0, 1]$	$[0, 20, 0, 0, 50]$

Auto-Regression: Masking

In what follows, we will consider only finite distributions over C real values.

Hence, we can model a conditional distribution with a mapping that maps a pair mask/known values to a distribution for the next value of the sequence:

$$f : \{0, 1\}^T \times \mathbb{R}^T \rightarrow \mathbb{R}^C$$

where the C output values can be either probabilities, or as we will prefer, logits.

This can be generalized beyond categorical distributions by mapping to parameters of any distribution.

Auto-Regression: Generation

In what follows, we will consider only finite distributions over $\mathbb{C}$ real values.

Hence, we can model a conditional distribution with a mapping that maps a pair mask/known values to a distribution for the next value of the sequence:

$$f : \{0, 1\}^T \times \mathbb{R}^T \rightarrow \mathbb{R}^{\mathbb{C}}$$

where the $\mathbb{C}$ output values can be either probabilities, or as we will prefer, logits.

This can be generalized beyond categorical distributions by mapping to parameters of any distribution.

Given such a model and a sampling function `sample`, the generative process for a full sequence is

$$x_1 \leftarrow \text{sample}(f(\{\}))$$

$$x_2 \leftarrow \text{sample}(f(\{X_1 = x_1\}))$$

$$x_3 \leftarrow \text{sample}(f(\{X_1 = x_1, X_2 = x_2\}))$$

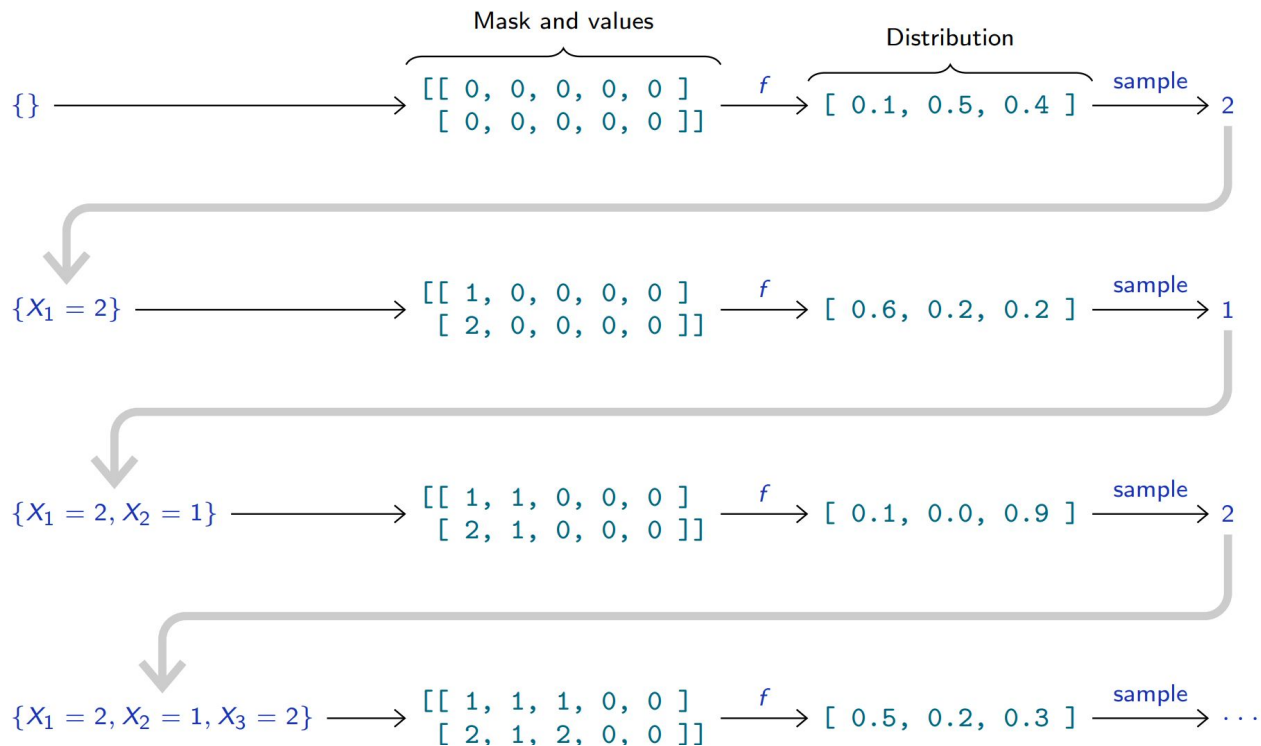
• • •

$$x_T \leftarrow \text{sample}(f(\{X_1 = x_1, X_2 = x_2, \dots, X_{T-1} = x_{T-1}\}))$$

A sampling procedure takes as input the probabilities (or logits) output by the model and outputs a value sampled randomly

Generation: Example

For instance, with $C = 3$ and $T = 5$:



Generation: Example

For instance, with $C = 3$ and $T = 5$:

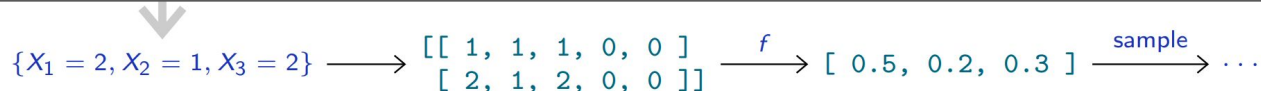
Mask and values Distribution sample

$[[0, 0, 0, 0, 0]]$ f $[0.5, 0.2, 0.3]$ sample

The package `torch.distributions` provides the necessary tools to sample from a variety of distributions.

```
lg = torch.tensor([ log(0.8), log(0.1), log(0.1) ])
dist = torch.distributions.categorical.Categorical(logits = lg)
s = dist.sample((10000,))
s.view(-1, 1) == torch.arange(3).view(1, -1).float().mean(0)
tensor([0.8037, 0.0988, 0.0975])
```

Sampling can also be done in batch



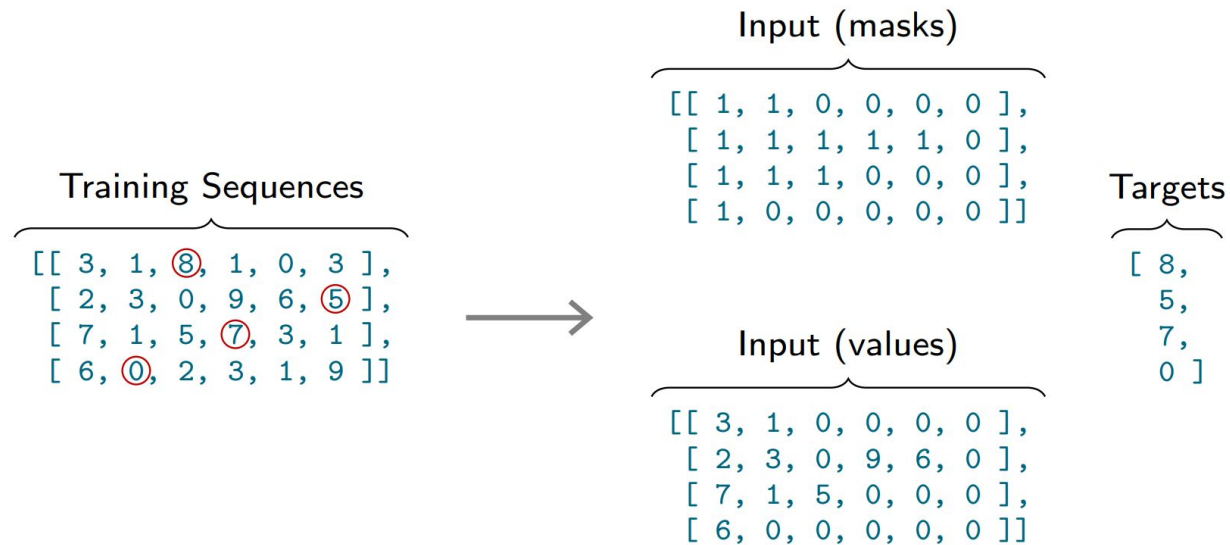
Auto-Regression: Training

With a finite distribution and the output values interpreted as logits, training consists of maximizing the likelihood of the training samples or minimizing cross-entropy loss

$$\begin{aligned}\mathcal{J}(f) &= - \sum_n \sum_t \log \hat{p}(X_t = x_{n,t} | X_1 = x_{n,1}, \dots, X_{t-1} = x_{n,t-1}) \\ &= \sum_n \sum_t \mathcal{L}(f((1, \dots, 1, 0, \dots, 0), (x_{n,1}, \dots, x_{n,t-1}, 0, \dots, 0)), x_{n,t})\end{aligned}$$

Auto-Regression: Training

In practice, for each batch, we sample a position to predict for each sample at random, from which we build the masks, conditioning values, and target values.

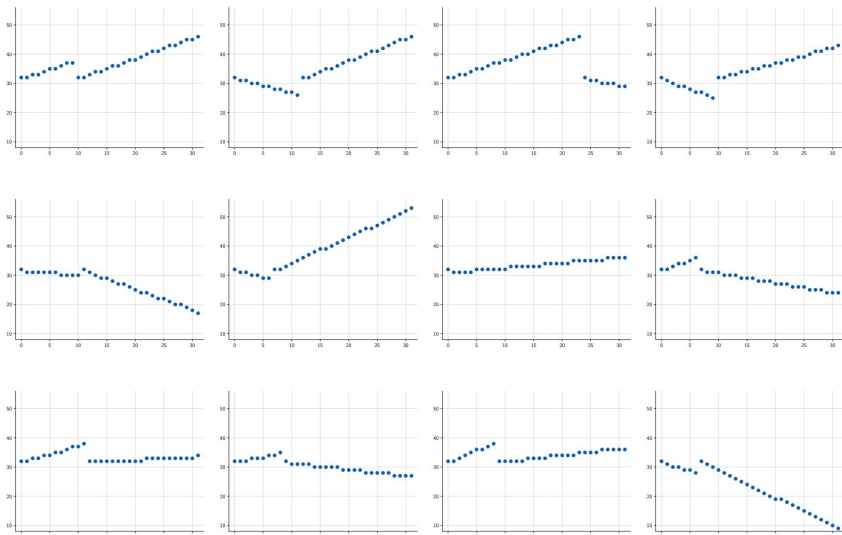


Example: Toy Problem

Each sequence is of length T , and take values in $\{0, \dots, 2T - 1\}$ hence $C = 2T$ different possible values at each point. A sequence starts at the middle value, $x_1 = T$.

The sequence behaves linearly with a slope in $[-1, 1]$ until a “cut” point at which it re-starts again at value T with a new slope in $[-1, 1]$.

This split point is chosen uniformly in $[1, T]$.



This example exhibits 2 interesting features:

- a macro structure (the cut), and
- at every point, a local affine structure.

We can work with a standard 1D convolutional network. The input to the network has two channels: one for the mask, and one for the value tensor.

The output of the model is a tensor of size target as shown earlier.

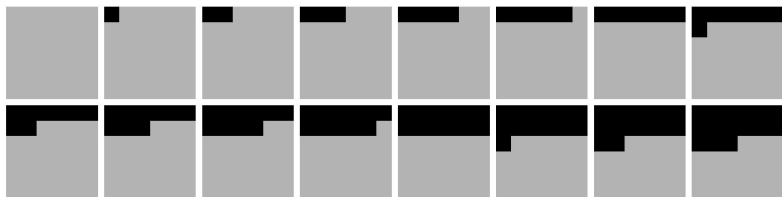
Auto-Regression: Images

The exact same auto-regressive approach generalizes to any tensor shape, as long as a visiting order of the coefficients is provided. For instance, for images, we can visit pixels in the “raster scan order” corresponding to the standard mapping in memory, top-to-bottom, left-to-right.



Auto-Regression: Images

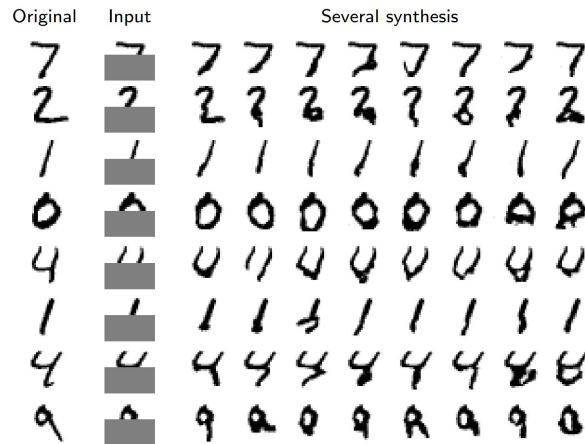
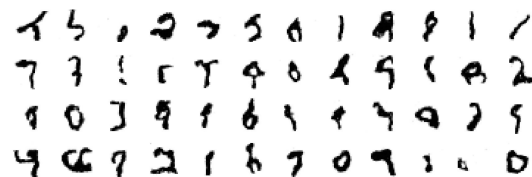
The exact same auto-regressive approach generalizes to any tensor shape, as long as a visiting order of the coefficients is provided. For instance, for images, we can visit pixels in the “raster scan order” corresponding to the standard mapping in memory, top-to-bottom, left-to-right.



Take LeNet with two input channels:

- one for the mask, which indicates the pixels already known, and
- the other for the actual pixel values

Here, the number of classes is the number of gray levels (from 0 to 255) that the component of the sequence can take.



Auto-Regression:

Some remarks:

- The **index ordering** for the sampling is a **design decision**. It can be fixed during train and test, or be **adaptive**.
- Even when there is a clear metric structure on the value space, best results are obtained with cross-entropy over a discretization of it.
 - This is due in large part to the ability of categorical distributions and cross-entropy to deal with exotic posteriors, in particular multi-modal.
- **Perplexity** is a metric used to evaluate the uncertainty or confidence of a probabilistic model in predicting the next value/token (e.g., word, phoneme, character).

If your model assigns: Word A: 0.7 | Word B: 0.2 | Word C: 0.1

Then it's less perplexed than if it assigns: Word A: 0.33 | Word B: 0.33 | Word C: 0.34

The flatter the distribution, the higher the perplexity.

$$\text{Perplexity} = \left(\prod_{t=1}^T \frac{1}{p(x_t | x_{<t})} \right)^{\frac{1}{T}}$$

Causal Convolutions

In autoregression, the input differs from one position to another:
when predicting a new component, both the mask and the value tensor are recomputed.

Consequently such a model does not leverage the fact that most computation is shared between positions.

Classical autoregressive models ensure that the distribution predicted for a certain component of the signal only depends on the component predicted before, and can compute a loss on all the sequence component in one pass.

These structures are called “causal”, since only the “past” can influence the “future”.

Causal Convolutions

Autoregression $f : \{0, 1\}^T \times \mathbb{R}^T \rightarrow \mathbb{R}^C$

Casual Computation $f : \mathbb{R}^T \rightarrow \mathbb{R}^{C \times T}$

Instead of predicting the distribution of one component, the model could predict the distributions at every position of the sequence.

where the 0s simply fill in for unknown values, and the mask is not needed.

$x_1 \leftarrow \text{sample}(f(\{\}))$
 $x_2 \leftarrow \text{sample}(f(\{X_1 = x_1\}))$
 $x_3 \leftarrow \text{sample}(f(\{X_1 = x_1, X_2 = x_2\}))$
• • •
 $x_T \leftarrow \text{sample}(f(\{X_1 = x_1, X_2 = x_2, \dots, X_{T-1} = x_{T-1}\}))$

$x_1 \leftarrow \text{sample}(f_1([0, \dots, 0]))$
 $x_2 \leftarrow \text{sample}(f_2([x_1, 0, \dots, 0]))$
 $x_3 \leftarrow \text{sample}(f_3([x_1, x_2, 0, \dots, 0]))$
• • •

Causal Convolutions

Casual Computation Loss is simplified if “future values” do not influence the prediction at a certain time.

Tremendous computational advantage during training f is computed once

$$\sum_t \mathcal{L}(f_t(x_1, \dots, x_{t-1}, 0, \dots, 0), x_t)$$
$$= \sum_t \mathcal{L}(f_t(x_1, \dots, x_T), x_t)$$

f is computed once

$$f_1([0, \dots, 0]) = f_1([x_1, \dots, x_T])$$

$$f_2([x_1, 0, \dots, 0]) = f_2([x_1, \dots, x_T])$$

$$f_3([x_1, x_2, 0, \dots, 0]) = f_3([x_1, \dots, x_T])$$

• • •

Causal Convolutions

Casual Computation Loss is simplified if “future values” do not influence the prediction at a certain time.

$$\sum_t \mathcal{L}(f_t(x_1, \dots, x_{t-1}, 0, \dots, 0), x_t) \\ = \sum_t \mathcal{L}(f_t(x_1, \dots, x_T), x_t)$$

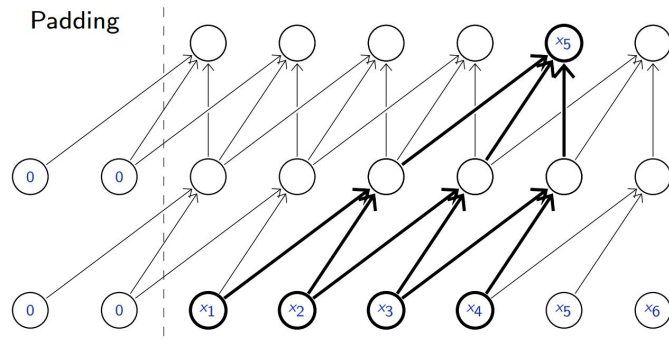
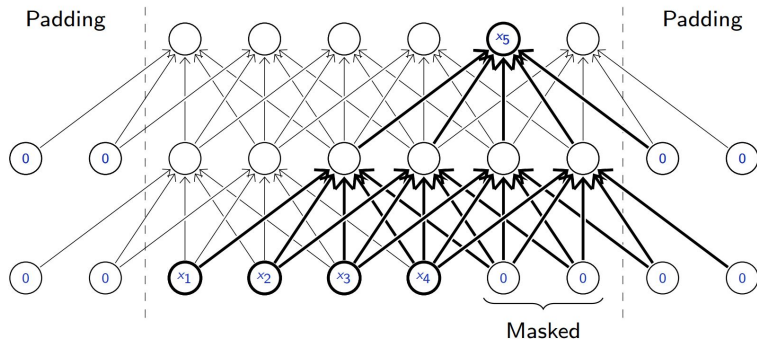
$$f_1([0, \dots, 0]) = f_1([x_1, \dots, x_T])$$

$$f_2([x_1, 0, \dots, 0]) = f_2([x_1, \dots, x_T])$$

$$f_3([x_1, x_2, 0, \dots, 0]) = f_3([x_1, \dots, x_T])$$

• • •

Tremendous computational advantage during training f is computed once



Causal Convolutions

Sometimes longer context is needed to capture global trends.

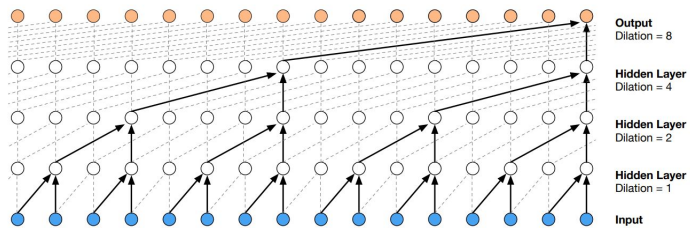
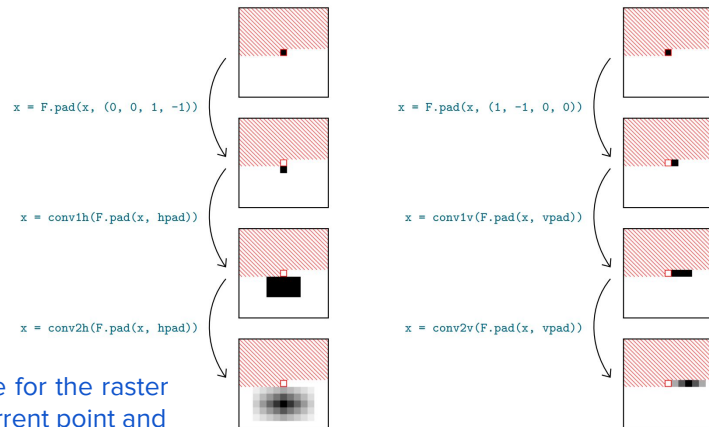
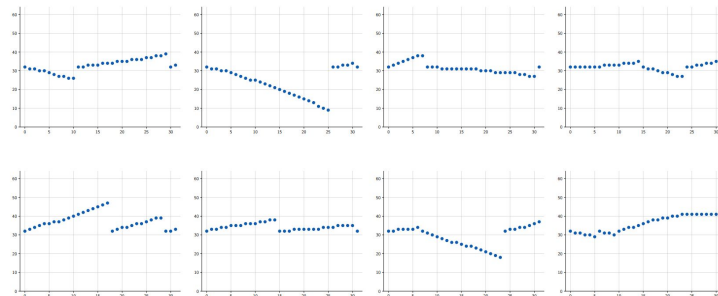


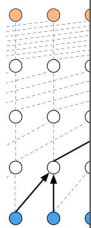
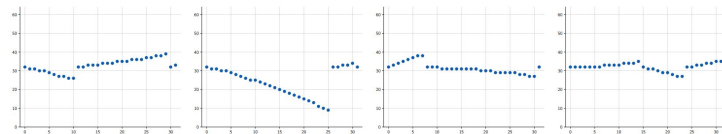
Figure 3: Visualization of a stack of *dilated* causal convolutional layers.



(Oord et al., 2016b) propose 2D convolutions that respect the causal structure for the raster scan order. This is achieved by combining convolutions that look above the current point and strictly on its left.

Causal Convolutions

Sometimes longer context is needed to capture global trends.



Such a fully convolutional model has no way to make the prediction position-dependent, which results here in local consistency, but fragmentation.

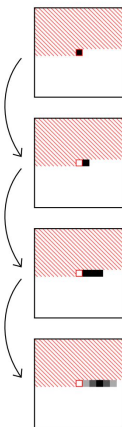
A classical fix is to supplement the input with a positional encoding, that is a multi-channel input that provides full information about the location.

Figure

A. Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu. WaveNet: A generative model for raw audio. CoRR, abs/1609.03499, 2016a.

A. Oord, N. Kalchbrenner, O. Vinyals, L. Espeholt, A. Graves, and K. Kavukcuoglu. Conditional image generation with PixelCNN decoders. CoRR, abs/1606.05328, 2016b.

(Oord et al., 2016b) propose 2D convolutions that respect the causal structure for the raster scan order. This is achieved by combining convolutions that look above the current point and strictly on its left.



Thanks

